Rapid Reaching With PD Controls

Candence Young
University of Massachussets Amherst
Amherst, MA
cadenceyoung@umass.edu

Marwan Hegab
University of Massachussets Amherst
Amherst, MA
mhegab@umass.edu

Caroline Zouloumian
Smith College
Northampton, MA
czouloumian@smith.edu

Madeline Gelnett
University of Massachussets Amherst
Amherst, MA
mgelnett@umass.edu

***Abstract*—This paper will detail the findings of how our team solved the Rapid Reaching Challenge in the MuJoCo physics simulator in Python. We will go over the method we used as our final solution including target selection, inverse kinematics, as well as Joint-Space Control and clipping. Then we will detail the challenges we faced building our program including environment building for the physics engine to work on one of the student's computers. After the challenges, we analyze our results on how our robot performed on different configurations of points and where we faced limitations. Finally we reflected on what we might do to improve our program such as utilizing reinforcement learning on the controls to make the robot run smoother.**

***Index Terms*—PD control, Jacobian, Pseudo Inverse, Coriolis and Centrifugal Forces**

I. Introduction

The Rapid Reaching Challenge is a task requiring one to move a robotic arm to a number of points whose positions are randomly generated in the shortest amount of time. This report will detail our team's findings and challenges to accomplish this goal through the MuJoCo physics engine.

II. Method

A. Overview

Our algorithm iteratively guides the robot's end-effector through each target. At each control step, the method selects the target by finding the nearest unreached point, using Euclidean distance. Once the target point is obtained, the algorithm computes the Cartesian position error between the current end-effector position and the target. Using the error, it computes the Jacobian of the robot arm to map from Cartesian space to joint space. To avoid instability near singularities, we employ the Damped Least Squares (DLS) method to solve the inverse kinematics, yielding the desired change in joint angles. Finally, the joint positions feed into a Proportional-Derivative (PD) controller, which outputs joint torques, supplemented by a bias term that compensates for gravitational, Coriolis, and centrifugal forces. The process repeats until the robot reaches every target position.

B. Target Selection

In our target selection method we used the points given to calculate the shortest path with Euclidean distance. First we find the position of the end effector. Once the end effector was found we used a greedy approach by solving for the Euclidean distance and creating a boolean mask array of distances. If a point has been reached at this stage then we mark this point as reached. From this boolean mask array we then check to see if all points were checked. If there were points still left unvisited by the end effector we set reached points to infinity and choose nearest neighbor utilizing the Euclidean distance formula as show in Figure 1.

$$d_i \;=\; \left\| \mathbf{x}_{\mathrm{EE}} - \mathbf{p}_i \right\| \;=\; \sqrt{(x_{\mathrm{EE}} - x_i)^2 + (y_{\mathrm{EE}} - y_i)^2 + (z_{\mathrm{EE}} - z_i)^2}.$$

Fig. 1. Euclidean distance used to pick the nearest target.

If all points have been reached then the program terminates as the goal is acquired.

C. Inverse Kinematics

Our original method did not include inverse kinematics and instead calculated torques directly using the Jacobian transpose and the idea of virtual work. We used the formula in Figure 2 to find our task torque at each control step. Although this method was able to reach the points somewhat effectively in 4-6 seconds, its movements were not very accurate. It would usually overshoot and oscillate during each movement and did not take direct paths. So, we decided to investigate using inverse kinematics to have a more direct goal, have more control over the robot's actions, and just explore how it would work differently. We chose to use the Damped Least Squares inverse because it allows for more stable control than other inverse kinematic solutions. While a traditional pseudoinverse Jacobian can have very small values at singularities causing impractically large velocity solutions, a Damped Least Squares inverse damps these large values and produces a continuous solution everywhere [1].

We used the formula in Figure 2 to calculate our change in joint position at each control step. We then tuned our α and λ values manually, testing which gave the best results for smoothness and speed.

$$\boxed{\Delta q \;=\; \alpha\, J^{\top}\Big(J\,J^{\top} + \lambda^{2} I\Big)^{-1} - q}$$

Fig. 2. Damped-Least-Squares (Levenberg–Marquardt) joint update used at every 1 kHz control step. α is the step-size, λ the damping factor.

Using inverse kinematics with DLS led to much more direct movement of our robot to each point without oscillations. We chose this method for our final solution.

D. Joint-Space Control and Clipping

Our final step was using basic PD control to calculate our joint torques. After getting desired joint positions from our inverse kinematics, we found the error with our current joint positions and multiplied by a Kp constant as our proportional term. Then we multiplied Kd by the current joint velocities and subtracted that as our derivative term. Then we added the bias forces provided by mujoco to compensate for gravity, coriolis, and centripetal forces. This gave our total equation for joint torques as show in Figure 3.

$$\boxed{\boldsymbol{\tau} \;=\; K_p\,(q_{\text{des}} - q) \;-\; K_d\,\dot{q} \;+\; \boldsymbol{\tau}_{\text{bias}}}$$

Fig. 3. Joint-space PD controller with model-based gravity/Coriolis compensation.

Finally, we clipped our torques based on the control range of each joint as a safety measure.

E. Challenges

There were multiple challenges through out our experiments with Mujoco as well as the dynamics and physics of our robot.

One of the challenges was trying to get Mujoco to run on one of our teammates computers. They kept getting the error "ImportError: You are running an x86_64 build of Python on an Apple Silicon machine. This is not supported by MuJoCo. Please install and run a native, arm64 build of Python". This was happening because the python that was running on the computer was a x86_64 build of Python when the it needed to rum an arm64 build of python to use MuJoCo. To solve this we downloaded anaconda and made a Conda environment as well as mini-forge. Then we ran the project in this environment and it worked.

Another challenge was the robot having jittery behavior, which happened in early attempts of control. Each joint would jitter back and forth and move in an unstable way. The torques of different links seemed to be all interacting and trying to compensate for each other. We found that one reason for this was just Kp values that were too high, causing the joints to constantly overshoot and re-correct. So, we set them overall lower and continued to adjust and experiment with the best values. Another reason was that there was a difference in the torque that different joints actually required. The shoulder pitch joint for example had to support a lot of weight, especially when horizontal, but the wrist joint did not. Setting the same Kp constant for all of them caused the lighter joints to have more torque than they needed. This caused them to rapidly flip back and forth which also caused instability in the other joints too. So, adjusting the torques between different joints, and tuning those parameters as well based on which joints seemed to be struggling against its forces versus which were moving too much, helped solve this issue.

Another challenge was compensating for gravity. At first we did not take it into consideration, and just set torques based on position error, but this caused the robot to sometimes not have enough force to overcome gravity, or it would just interfere in general. We fixed this by simply adding the bias forces mujoco provides.

Our last challenge was the oscillation of the pointer around the point. Instead of directly going to the point, the pointer would make several attempts of getting very close to the point, but not on the point. We solved this by tuning our parameters in an iterative trial-and-error method.

III. Result and Analysis

Overall, our Rapid Reaching algorithm performs well in the scope of the class. The robot arm reaches all the point with an overall mean completion time of 3.92 seconds, placing us at the third place in the competition, out of seven teams.

Seed	123	403	000	111	222	Mean
Seconds	3.45	3.96	3.86	3.95	4.39	3.92

Fig. 4. Our group's results in the class competition tests.

Figure 4 presents our group's results in the class competition tests. Each time value is the mean of 3 trials with respect to the given seed.

In contrary to our earlier stages of determining the best solution, the arm goes directly towards each point without oscillating, and picks the closest point first. It runs quickly and smoothly.

IV. Conclusion

A. Limitations

The arm can get stuck on certain points for longer than it is expected to. These edge cases happen in two cases. If the trajectory of the arm crosses the floor, the arm will get stuck. If the point is at the maximum reach for the arm, the arm will take more time to reach it.

B. Future Work

Future work includes researching techniques enabling the arm not to get stuck in our two edge cases. We would like to look for and compare algorithms to calculate paths with least resistance, enabling the controller to move smoothly. To obtain an arched, smoother movement, we also aim to implement forward kinematics. As better controls could make the arm more efficient, our control code is always a work in progress.

Another area of improvement could be the use of reinforcement learning. We'd use reinforcement to train the

controls to work better in different scenarios. One of the most effective methods of training model in relation to a study of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) is the Deep Deterministic Policy Gradient Algorithm[2]. This model has a Q-value function as a critic and as a deterministic policy actor[3]. This model was trained against other models like the Vanilla Policy Gradient in the task called Random Target Reaching. This experiment was very similar, if not exactly like our rapid-reaching challenge. For our reward function, we would use things like reaching a point, reaching a point never reached before, and if it's reached all the points as fast or faster than the previous iterations. The penalties we'd use for excessive torque and if it's not reached all the points.

Throughout the final presentations of this challenge there were many other groups who chose the rapid reaching problem. With this, we've taken inspiration and wish to look into some of their methods to improve our strategy of going point to point. One of the most prevalent ideas that we think would be more efficient with finding points in terms of distance is the Traveling salesman algorithm. This is because the shortest distance between the current point is not always the most optimal approach but traveling salesman has proved to find the shortest path. With this we can preplan our path of points to make it faster and more efficient during running time.

With these changes we hope to improve your robot's forward dynamics, it's controls with reinforcement learning, as well as it's shortest path finding approach through the the traveling salesman algorithm.

ACKNOWLEDGMENT

We would like to thank Professor Kim Donghyun for this semester in COMPSCI403: Introduction to Robotics as well as the TA's Nisal Minula Perera Kankanige, Qianhuang Li, and Shangqun (Simon) Yu.

REFERENCES

[1] C. W. Wampler, "Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods," in IEEE Transactions on Systems, Man, and Cybernetics, vol. 16, no. 1, pp. 93-101, Jan. 1986, doi: 10.1109/TSMC.1986.289285.

[2] Andrea Franceschetti, Elisa Tosello, Nicola Castaman, Stefano Ghidoni(2018), Robotic Arm Control and Task Training through Deep Reinforcement Learning;Submitted to IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) 2018

[3] Deep Deterministic Policy Gradient (DDPG) Agent ,https://www.mathworks.com/help/reinforcement-learning/ug/ddpg-agents.html

[4] Diego Gangl, Calculating distances in Blender with Python, Sinestesia: https://sinestesia.co/blog/tutorials/calculating-distances-in-blender-with-python/

[5] T. Sauer, *Numerical Analysis*, Third Edition. Chapter 4.5. Boston, MA: Pearson, 2018. Available: https://www.pearson.com/en-us/subject-catalog/p/numerical-analysis/